List of Experiments

Total No of Labs: 14

S. No.	Programs List	No. of Labs
CAD Experiments		
1	Study of basic graphics functions defined in “graphics.h”.	1
2	To implement Digital differential analyzer (DDA) Algorithm for drawing a line segment between two given end points A (x1, y1) and B(x2, y2).	1
3	To implement Bresenham’s line drawing algorithm for drawing a line segment between two given endpoints A (x1, y2) and B(x2, y2).	1
4	To implement Bresenham’s Cicrcle drawing algorithm for drawing a circle with a given center of circle P(x, y) and radius r.	1
5	To apply the basic 2D transformations such as translation, Scaling, Rotation, shearing and reflection for a given 2D object.	2
6	To apply the basic transformations such as translation, Scaling, Rotation for a given 3D object.	2
7	To generate a smooth curve by using Bezier curve technique for a given set of control points.	1
8	To implement Cohen–Sutherland 2D clipping and window–viewport mapping	1
Product Design		
9	Introduction to CAD software’s.	1
10	Generation of any one simple solid model using any CAD software.	1
11	Generation of any one assembly of mechanical part using any CAD software.	1
12	Design and develop the complete lifecycle for any product.	1

Experiment No. – 6

Aim: To apply the basic transformations such as translation, Scaling, Rotation for a given 3D object.

Theory:

Translation

In three-dimensional homogeneous coordinates, a point translates from $P(x, y, z, 1)$ to $P_T(x_T, y_T, z_T, 1)$ using the matrix equation

$$\begin{Bmatrix} x_T \\ y_T \\ z_T \\ 1 \end{Bmatrix} = [T_t].\begin{Bmatrix} x \\ y \\ z \\ 1 \end{Bmatrix} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}.\begin{Bmatrix} x \\ y \\ z \\ 1 \end{Bmatrix}$$

Rotation

Three-dimensional z-axis rotation equation in matrix form is expressed as

$$[T_{rz}] = \begin{bmatrix} \cos\alpha & -\sin\alpha & 0 & 0 \\ \sin\alpha & \cos\alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Three-dimensional x -axis rotation equation in matrix form is expressed as

$$[T_{rx}] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\beta & -\sin\beta & 0 \\ 0 & \sin\beta & \cos\beta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Three-dimensional y-axis rotation equation in matrix form is expressed as

$$[T_{ry}] = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Scaling

$$\begin{Bmatrix} x_T \\ y_T \\ z_T \\ 1 \end{Bmatrix} = [S].\begin{Bmatrix} x \\ y \\ z \\ 1 \end{Bmatrix} = \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.\begin{Bmatrix} x \\ y \\ z \\ 1 \end{Bmatrix}$$

Program for translation:

```
#include<stdio.h>
#include<conio.h>
```

```
#include<math.h>
#include<process.h>
#include<graphics.h>
int x1,x2,y1,y2,mx,my,depth;
void draw();
void trans();
void main()
{
int gd=DETECT,gm,c;
initgraph(&gd,&gm,"C:\\TurboC3\\BGI");
printf("\n\t\t3D Translation\n\n");
printf("\nEnter 1st top value(x1,y1):");
scanf("%d%d",&x1,&y1);
printf("Enter right bottom value(x2,y2):");
scanf("%d%d",&x2,&y2);
depth=(x2-x1)/4;
mx=(x1+x2)/2;
my=(y1+y2)/2;
draw();
getch();
cleardevice();
trans();
getch();
}
void draw()
{
bar3d(x1,y1,x2,y2,depth,1);
}
void trans()
{
int a1,a2,b1,b2,dep,x,y;
printf("\n Enter the Translation Distances:");
scanf("%d%d",&x,&y);
a1=x1+x;
a2=x2+x;
b1=y1+y;
b2=y2+y;
dep=(a2-a1)/4;
bar3d(a1,b1,a2,b2,dep,1);
setcolor(5);
draw();
}
```

Program for Rotation:

```
#include<stdio.h>
#include<conio.h>
#include<math.h>
#include<graphics.h>
int x1,x2,y1,y2,mx,my,depth;
void draw();
void rotate();
```

```
void main()
{
int gd=DETECT,gm,c;
initgraph(&gd,&gm,"C:\\TurboC3\\BGI");
printf("\n3D Transformation Rotating\n\n");
printf("\nEnter 1st top value(x1,y1):");
scanf("%d%d",&x1,&y1);
printf("Enter right bottom value(x2,y2):");
scanf("%d%d",&x2,&y2);
depth=(x2-x1)/4;
mx=(x1+x2)/2;
my=(y1+y2)/2;
draw(); getch();
cleardevice();
rotate();
getch();
}
void draw()
{
bar3d(x1,y1,x2,y2,depth,1);
}
void rotate()
{
float t;
int a1,b1,a2,b2,dep;
printf("Enter the angle to rotate=");
scanf("%f",&t);
t=t*(3.14/180);
a1=mx+(x1-mx)*cos(t)-(y1-my)*sin(t);
a2=mx+(x2-mx)*cos(t)-(y2-my)*sin(t);
b1=my+(x1-mx)*sin(t)-(y1-my)*cos(t);
b2=my+(x2-mx)*sin(t)-(y2-my)*cos(t);
if(a2>a1)
dep=(a2-a1)/4;
else
dep=(a1-a2)/4;
bar3d(a1,b1,a2,b2,dep,1); setcolor(5);
}
```

Program for scaling:

```
#include<stdio.h>
#include<conio.h>
#include<math.h>
#include<process.h>
#include<graphics.h>
int x1,x2,y1,y2,mx,my,depth;
void draw();
void scale();
void main()
{
int gd=DETECT,gm,c;
```

```
initgraph(&gd,&gm,"C:\\TurboC3\\BGI");
printf("\n\t\t3D Scaling\n\n");
printf("\nEnter 1st top value(x1,y1):");
scanf("%d%d",&x1,&y1);
printf("Enter right bottom value(x2,y2):");
scanf("%d%d",&x2,&y2);
depth=(x2-x1)/4;
mx=(x1+x2)/2;
my=(y1+y2)/2;
draw();
getch();
cleardevice();
scale();
getch();
}
void draw()
{
bar3d(x1,y1,x2,y2,depth,1);
}
void scale()
{
int x,y,a1,a2,b1,b2,dep;
printf("\n\n Enter scaling Factors:");
scanf("%d%d",&x,&y);
a1=mx+(x1-mx)*x;
a2=mx+(x2-mx)*x;
b1=my+(y1-my)*y;
b2=my+(y2-my)*y;
dep=(a2-a1)/4;
bar3d(a1,b1,a2,b2,dep,1);
setcolor(5);
draw();
}
```